

Conexión de React, Express y SQL Server

Proyecto: CRUDEjercicio

Objetivo: documentar el código modificado para enviar alumnos desde React, guardarlos con Express en SQL Server y consultar los registros para mostrarlos en el frontend.

Aclaración sobre 'sin APIs'

El frontend no puede acceder directamente a SQL Server de forma segura. En este proyecto sí existe una API local creada con Express. No se utilizó una API externa ni un servicio de terceros: React se comunica con las rutas HTTP GET y POST de Express en /api/alumnos.

Arquitectura resultante

```

Navegador (React / Vite, puerto 5173)
  | fetch('/api/alumnos')
  v
Proxy de Vite: /api -> http://localhost:3000
  |
  v
Backend Express (puerto 3000)
  | consultas parametrizadas con mssql
  v
SQL Server (puerto 1433) - base Universidad - tabla Alumnos
  
```

Resumen de cambios

- vite-project/vite.config.js: se añadió un proxy para redirigir las solicitudes que comienzan con /api al backend de Express.
- vite-project/src/Componentes/Form.jsx: se añadieron estados controlados, validaciones y la petición POST que envía el nuevo alumno.
- vite-project/src/App.jsx: se añadió la petición GET, el control de carga y errores, y una tabla para mostrar los alumnos.
- APIS/server.js ya contenía las rutas GET y POST. React ahora consume esas rutas; Express es la capa que consulta SQL Server.

1. Proxy de desarrollo de Vite

Este cambio permite escribir fetch('/api/alumnos') en React. Vite recibe la solicitud en el puerto del frontend y, durante el desarrollo, la reenvía a Express en el puerto 3000. Así se evitan direcciones repetidas y problemas de CORS mientras ambos servicios se ejecutan localmente.

Código actual de vite-project/vite.config.js:

```

import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': 'http://localhost:3000',
    },
  },
})
  
```

2. Formulario y envío de datos

El formulario usa `useState` para conservar los valores escritos. Cada input tiene un atributo `name` igual al nombre que espera Express: Nombre, Apellido, Carnet, Carrera y Edad.

Al enviar, `event.preventDefault()` evita recargar la página. `fetch` realiza un POST, `Content-Type` indica que el cuerpo es JSON y `JSON.stringify` convierte el objeto de JavaScript en el texto que Express puede procesar con `express.json()`.

Cuando el backend responde correctamente, el formulario se limpia y `onAlumnoCreado` solicita nuevamente la lista para reflejar el registro recién insertado.

Código actual de `vite-project/src/Componentes/Form.jsx`:

```
import { useState } from "react";

const datosIniciales = {
  Nombre: "",
  Apellido: "",
  Carnet: "",
  Carrera: "",
  Edad: "",
};

function Form({ onAlumnoCreado }) {
  const [datos, setDatos] = useState(datosIniciales);
  const [confirmado, setConfirmado] = useState(false);
  const [enviando, setEnviando] = useState(false);
  const [mensaje, setMensaje] = useState(null);

  const actualizarCampo = (event) => {
    const { name, value } = event.target;
    const nuevoValor = name === "Edad" ? value.replace(/\D/g, "").slice(0, 3) : value;

    setDatos((datosActuales) => ({
      ...datosActuales,
      [name]: nuevoValor,
    }));
  };

  const enviarFormulario = async (event) => {
    event.preventDefault();
    setEnviando(true);
    setMensaje(null);

    try {
      const respuesta = await fetch("/api/alumnos", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          ...datos,
          Edad: Number(datos.Edad),
        }),
      });

      const resultado = await respuesta.json();

      if (!respuesta.ok) {
        throw new Error(resultado.mensaje || "No se pudo guardar el alumno");
      }

      setDatos(datosIniciales);
      setConfirmado(false);
      setMensaje({ tipo: "success", texto: resultado.mensaje });
    }
  };
}
```

```

    await onAlumnoCreado?();
  } catch (error) {
    setMensaje({ tipo: "danger", texto: error.message });
  } finally {
    setEnviando(false);
  }
};

return (
  <form onSubmit={enviarFormulario}>
    <div className="mb-3">
      <label htmlFor="nombre" className="form-label">
        Nombre
      </label>
      <input
        type="text"
        className="form-control"
        id="nombre"
        name="Nombre"
        value={datos.Nombre}
        onChange={actualizarCampo}
        maxLength={50}
        required
      />
    </div>
    <div className="mb-3">
      <label htmlFor="apellido" className="form-label">
        Apellido
      </label>
      <input
        type="text"
        className="form-control"
        id="apellido"
        name="Apellido"
        value={datos.Apellido}
        onChange={actualizarCampo}
        maxLength={50}
        required
      />
    </div>
    <div className="mb-3">
      <label htmlFor="carnet" className="form-label">
        Carnet
      </label>
      <input
        type="text"
        className="form-control"
        id="carnet"
        name="Carnet"
        value={datos.Carnet}
        onChange={actualizarCampo}
        maxLength={20}
        required
      />
    </div>
    <div className="mb-3">
      <label htmlFor="carrera" className="form-label">
        Carrera
      </label>
      <input
        type="text"
        className="form-control"
        id="carrera"
        name="Carrera"
        value={datos.Carrera}
        onChange={actualizarCampo}
        maxLength={100}
      />
    </div>
  </form>
);

```

```

      required
    />
  </div>
  <div className="mb-3">
    <label htmlFor="edad" className="form-label">
      Edad
    </label>
    <input
      type="text"
      inputMode="numeric"
      maxLength={3}
      pattern="[0-9]{1,3}"
      className="form-control"
      id="edad"
      name="Edad"
      value={datos.Edad}
      onChange={actualizarCampo}
      style={{ width: "6rem" }}
      title="Ingresa una edad de máximo 3 dígitos"
      required
    />
  </div>
  <div className="mb-3 form-check">
    <input
      type="checkbox"
      className="form-check-input"
      id="confirmacion"
      checked={confirmado}
      onChange={(event) => setConfirmado(event.target.checked)}
      required
    />
    <label className="form-check-label" htmlFor="confirmacion">
      Confirmo que estos datos son correctos
    </label>
  </div>

  {mensaje && (
    <div className={`alert alert-${mensaje.tipo}`} role="alert">
      {mensaje.texto}
    </div>
  )}

  <button type="submit" className="btn btn-primary" disabled={enviando}>
    {enviando ? "Guardando..." : "Guardar alumno"}
  </button>
</form>
);
}

export default Form;

```

3. Consulta y presentación de alumnos

App.jsx ejecuta GET /api/alumnos al abrir la página. La respuesta JSON se guarda en el estado alumnos y map genera una fila por cada registro. También se manejan estados de carga y error.

La función cargarAlumnos se comparte con el botón Actualizar y con Form. Esto evita duplicar la lógica y actualiza la tabla después de un POST exitoso.

Código actual de vite-project/src/App.jsx:

```

import { useCallback, useEffect, useState } from "react";
import Form from "../Componentes/Form";

const obtenerAlumnos = async () => {

```

```
const respuesta = await fetch("/api/alumnos");
const resultado = await respuesta.json();

if (!respuesta.ok) {
  throw new Error(resultado.mensaje || "No se pudieron obtener los alumnos");
}

return resultado;
};

function App() {
  const [alumnos, setAlumnos] = useState([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState("");

  const cargarAlumnos = useCallback(async () => {
    setCargando(true);
    setError("");

    try {
      setAlumnos(await obtenerAlumnos());
    } catch (errorPeticion) {
      setError(errorPeticion.message);
    } finally {
      setCargando(false);
    }
  }, []);

  useEffect(() => {
    let componenteActivo = true;

    obtenerAlumnos()
      .then((resultado) => {
        if (componenteActivo) setAlumnos(resultado);
      })
      .catch((errorPeticion) => {
        if (componenteActivo) setError(errorPeticion.message);
      })
      .finally(() => {
        if (componenteActivo) setCargando(false);
      });

    return () => {
      componenteActivo = false;
    };
  }, []);

  return (
    <main className="container py-4">
      <h1 className="mb-4">Registro de alumnos</h1>

      <div className="row g-5">
        <section className="col-lg-5">
          <h2 className="h4 mb-3">Nuevo alumno</h2>
          <Form onAlumnoCreado={cargarAlumnos} />
        </section>

        <section className="col-lg-7">
          <div className="d-flex justify-content-between align-items-center mb-3">
            <h2 className="h4 mb-0">Alumnos registrados</h2>
            <button
              type="button"
              className="btn btn-outline-secondary btn-sm"
              onClick={cargarAlumnos}
              disabled={cargando}
            >
              Actualizar
            </button>
          </div>
          <div>
            <table>
              <thead>
                <tr>
                  <th>Nombre</th>
                  <th>Apellido</th>
                  <th>Edad</th>
                </tr>
              </thead>
              <tbody>
                <tr>
                  <td>Juan</td>
                  <td>Perez</td>
                  <td>25</td>
                </tr>
                <tr>
                  <td>Maria</td>
                  <td>Garcia</td>
                  <td>30</td>
                </tr>
                <tr>
                  <td>Carlos</td>
                  <td>Rodriguez</td>
                  <td>28</td>
                </tr>
                <tr>
                  <td>Ana</td>
                  <td>Lopez</td>
                  <td>22</td>
                </tr>
                <tr>
                  <td>Pedro</td>
                  <td>Martinez</td>
                  <td>35</td>
                </tr>
                <tr>
                  <td>Sofia</td>
                  <td>Hernandez</td>
                  <td>27</td>
                </tr>
                <tr>
                  <td>Diego</td>
                  <td>Vargas</td>
                  <td>32</td>
                </tr>
                <tr>
                  <td>Valeria</td>
                  <td>Torres</td>
                  <td>29</td>
                </tr>
                <tr>
                  <td>Alejandro</td>
                  <td>Castro</td>
                  <td>24</td>
                </tr>
                <tr>
                  <td>Isabella</td>
                  <td>Munoz</td>
                  <td>31</td>
                </tr>
                <tr>
                  <td>Luis</td>
                  <td>Romero</td>
                  <td>26</td>
                </tr>
                <tr>
                  <td>Gabriela</td>
                  <td>Gonzalez</td>
                  <td>33</td>
                </tr>
                <tr>
                  <td>Sebastian</td>
                  <td>Nieto</td>
                  <td>23</td>
                </tr>
                <tr>
                  <td>Camila</td>
                  <td>Flores</td>
                  <td>34</td>
                </tr>
                <tr>
                  <td>Mateo</td>
                  <td>Rios</td>
                  <td>21</td>
                </tr>
                <tr>
                  <td>Valentina</td>
                  <td>Cruz</td>
                  <td>36</td>
                </tr>
                <tr>
                  <td>Javier</td>
                  <td>Vega</td>
                  <td>20</td>
                </tr>
                <tr>
                  <td>Mariana</td>
                  <td>Mendez</td>
                  <td>37</td>
                </tr>
                <tr>
                  <td>Nicolas</td>
                  <td>Cabrera</td>
                  <td>19</td>
                </tr>
                <tr>
                  <td>Alejandra</td>
                  <td>Soto</td>
                  <td>38</td>
                </tr>
                <tr>
                  <td>Felipe</td>
                  <td>Lima</td>
                  <td>18</td>
                </tr>
                <tr>
                  <td>Sofia</td>
                  <td>Garcia</td>
                  <td>39</td>
                </tr>
                <tr>
                  <td>Diego</td>
                  <td>Rodriguez</td>
                  <td>17</td>
                </tr>
                <tr>
                  <td>Valeria</td>
                  <td>Hernandez</td>
                  <td>40</td>
                </tr>
                <tr>
                  <td>Luis</td>
                  <td>Torres</td>
                  <td>16</td>
                </tr>
                <tr>
                  <td>Gabriela</td>
                  <td>Munoz</td>
                  <td>41</td>
                </tr>
                <tr>
                  <td>Sebastian</td>
                  <td>Romero</td>
                  <td>15</td>
                </tr>
                <tr>
                  <td>Camila</td>
                  <td>Gonzalez</td>
                  <td>42</td>
                </tr>
                <tr>
                  <td>Mateo</td>
                  <td>Rios</td>
                  <td>14</td>
                </tr>
                <tr>
                  <td>Valentina</td>
                  <td>Cruz</td>
                  <td>43</td>
                </tr>
                <tr>
                  <td>Javier</td>
                  <td>Vega</td>
                  <td>13</td>
                </tr>
                <tr>
                  <td>Mariana</td>
                  <td>Mendez</td>
                  <td>44</td>
                </tr>
                <tr>
                  <td>Nicolas</td>
                  <td>Cabrera</td>
                  <td>12</td>
                </tr>
                <tr>
                  <td>Alejandra</td>
                  <td>Soto</td>
                  <td>45</td>
                </tr>
                <tr>
                  <td>Felipe</td>
                  <td>Lima</td>
                  <td>11</td>
                </tr>
                <tr>
                  <td>Sofia</td>
                  <td>Garcia</td>
                  <td>46</td>
                </tr>
                <tr>
                  <td>Diego</td>
                  <td>Rodriguez</td>
                  <td>10</td>
                </tr>
                <tr>
                  <td>Valeria</td>
                  <td>Hernandez</td>
                  <td>47</td>
                </tr>
                <tr>
                  <td>Luis</td>
                  <td>Torres</td>
                  <td>9</td>
                </tr>
                <tr>
                  <td>Gabriela</td>
                  <td>Munoz</td>
                  <td>48</td>
                </tr>
                <tr>
                  <td>Sebastian</td>
                  <td>Romero</td>
                  <td>8</td>
                </tr>
                <tr>
                  <td>Camila</td>
                  <td>Gonzalez</td>
                  <td>49</td>
                </tr>
                <tr>
                  <td>Mateo</td>
                  <td>Rios</td>
                  <td>7</td>
                </tr>
                <tr>
                  <td>Valentina</td>
                  <td>Cruz</td>
                  <td>50</td>
                </tr>
                <tr>
                  <td>Javier</td>
                  <td>Vega</td>
                  <td>6</td>
                </tr>
                <tr>
                  <td>Mariana</td>
                  <td>Mendez</td>
                  <td>51</td>
                </tr>
                <tr>
                  <td>Nicolas</td>
                  <td>Cabrera</td>
                  <td>5</td>
                </tr>
                <tr>
                  <td>Alejandra</td>
                  <td>Soto</td>
                  <td>52</td>
                </tr>
                <tr>
                  <td>Felipe</td>
                  <td>Lima</td>
                  <td>4</td>
                </tr>
                <tr>
                  <td>Sofia</td>
                  <td>Garcia</td>
                  <td>53</td>
                </tr>
                <tr>
                  <td>Diego</td>
                  <td>Rodriguez</td>
                  <td>3</td>
                </tr>
                <tr>
                  <td>Valeria</td>
                  <td>Hernandez</td>
                  <td>54</td>
                </tr>
                <tr>
                  <td>Luis</td>
                  <td>Torres</td>
                  <td>2</td>
                </tr>
                <tr>
                  <td>Gabriela</td>
                  <td>Munoz</td>
                  <td>55</td>
                </tr>
                <tr>
                  <td>Sebastian</td>
                  <td>Romero</td>
                  <td>1</td>
                </tr>
                <tr>
                  <td>Camila</td>
                  <td>Gonzalez</td>
                  <td>56</td>
```

```

    </button>
  </div>

  {error && <div className="alert alert-danger">{error}</div>}

  {cargando ? (
    <p>Cargando alumnos...</p>
  ) : alumnos.length === 0 ? (
    <p>No hay alumnos registrados.</p>
  ) : (
    <div className="table-responsive">
      <table className="table table-striped table-bordered align-middle">
        <thead>
          <tr>
            <th>ID</th>
            <th>Nombre</th>
            <th>Apellido</th>
            <th>Carnet</th>
            <th>Carrera</th>
            <th>Edad</th>
          </tr>
        </thead>
        <tbody>
          {alumnos.map((alumno) => (
            <tr key={alumno.IdAlumno}>
              <td>{alumno.IdAlumno}</td>
              <td>{alumno.Nombre}</td>
              <td>{alumno.Apellido}</td>
              <td>{alumno.Carnet}</td>
              <td>{alumno.Carrera}</td>
              <td>{alumno.Edad}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )}
</section>
</div>
</main>
);
}

export default App;

```

4. Backend Express

Express recibe las solicitudes enviadas por React. `express.json()` interpreta el cuerpo JSON del POST. La ruta GET ejecuta SELECT y devuelve `result.recordset`; la ruta POST usa parámetros tipados de mssql y ejecuta INSERT.

Las entradas parametrizadas reducen el riesgo de inyección SQL porque los datos no se concatenan directamente en la consulta.

Código de APIS/server.js que atiende al frontend:

```

const express = require("express");
const { sql, poolPromise } = require("../db");

const app = express();

// Permite recibir información en formato JSON
app.use(express.json());

// =====
// MÉTODO GET

```

```

// Consultar todos los alumnos
// =====

app.get("/api/alumnos", async (req, res) => {
  try {
    // Obtener conexión a SQL Server
    const pool = await poolPromise;

    // Ejecutar SELECT
    const result = await pool.request().query(`
      SELECT
        IdAlumno,
        Nombre,
        Apellido,
        Carnet,
        Carrera,
        Edad
      FROM Alumnos
    `);

    // Devolver resultado en formato JSON
    res.json(result.recordset);
  } catch (error) {
    console.error("Error al consultar:", error);

    res.status(500).json({
      mensaje: "Error al consultar los alumnos",
      error: error.message,
    });
  }
});

// =====
// MÉTODO POST
// Insertar un nuevo alumno
// =====

app.post("/api/alumnos", async (req, res) => {
  try {
    // Recibir datos enviados desde Postman
    const { Nombre, Apellido, Carnet, Carrera, Edad } = req.body;

    // Obtener conexión a SQL Server
    const pool = await poolPromise;

    // Ejecutar INSERT
    await pool
      .request()

      .input("Nombre", sql.VarChar(50), Nombre)
      .input("Apellido", sql.VarChar(50), Apellido)
      .input("Carnet", sql.VarChar(20), Carnet)
      .input("Carrera", sql.VarChar(100), Carrera)
      .input("Edad", sql.Int, Edad).query(`
        INSERT INTO Alumnos
        (
          Nombre,
          Apellido,
          Carnet,
          Carrera,
          Edad
        )
        VALUES
        (
          @Nombre,
          @Apellido,
          @Carnet,

```

```

        @Carrera,
        @Edad
    )
    `);

// Respuesta al cliente
res.status(201).json({
  mensaje: "Alumno insertado correctamente",
});
} catch (error) {
  console.error("Error al insertar:", error);

  res.status(500).json({
    mensaje: "Error al insertar el alumno",
    error: error.message,
  });
}
});

// =====
// INICIAR SERVIDOR
// =====

const PORT = 3000;

app.listen(PORT, () => {
  console.log(`Servidor ejecutándose en http://localhost:${PORT}`);
});

```

5. Conexión con SQL Server

APIS/db.js crea y reutiliza un pool de conexiones con mssql. server.js espera poolPromise antes de ejecutar cada consulta. Por seguridad, la contraseña no se reproduce en este documento.

Configuración documentada con contraseña ocultada:

```

const sql = require("mssql");

const config = {
  user: "sa",
  password: "*****",
  server: "localhost",
  database: "Universidad",
  options: {
    encrypt: false,
    trustServerCertificate: true,
  },
};

const poolPromise = new sql.ConnectionPool(config)
  .connect()
  .then((pool) => {
    console.log(" Conectado correctamente a SQL Server");
    return pool;
  })
  .catch((error) => {
    console.error(" Error conectando a SQL Server:");
    console.error(error);
    throw error;
  });

module.exports = {
  sql,
  poolPromise,
};

```


6. Flujo completo

Guardar un alumno:

- 1. El usuario completa Form y presiona Guardar alumno.
- 2. React envía POST /api/alumnos con un objeto JSON.
- 3. El proxy de Vite reenvía la petición a localhost:3000.
- 4. Express lee req.body y ejecuta INSERT INTO Alumnos mediante mssql.
- 5. Express responde con HTTP 201 y un mensaje de éxito.
- 6. React vuelve a ejecutar GET /api/alumnos y actualiza la tabla.

Consultar alumnos:

- 1. Al cargar App, React envía GET /api/alumnos.
- 2. Express ejecuta SELECT sobre la tabla Alumnos.
- 3. SQL Server devuelve las filas y Express las responde como JSON.
- 4. React guarda el arreglo recibido y lo presenta mediante alumnos.map().

7. Ejecución y puertos

```
# Terminal 1: backend
cd APIS
node server.js

# Terminal 2: frontend
cd vite-project
npm run dev
```

Puertos: SQL Server usa 1433, Express usa 3000 y Vite normalmente usa 5173. Si se modifica vite.config.js mientras Vite está abierto, es necesario reiniciarlo.

Resultado de validación

El frontend compiló correctamente con npm run build, el linter terminó sin advertencias ni errores y se comprobó que SQL Server permite consultar la tabla Alumnos.